0 · พื้นฐาน OOP (คำนิยามที่ชอบถาม)

OOP = เขียนโปรแกรมโดยมองเป็น**วัตถุ**ที่มีข้อมูล+พฤติกรรม ต่างจาก procedural ที่มองเป็นลำดับขั้นตอน

Class = แม่แบบ/พิมพ์เขียว กำหนด attribute + method · **Object/Instance** = สิ่งที่สร้างจาก class ด้วย `new` มีค่าของตัวเอง (1 class → หลาย object)

Attribute / Field / Property = ข้อมูล (สิ่งที่มี) · **Method** = พฤติกรรม (สิ่งที่ทำได้) · **State** = ค่า attribute ณ ขณะหนึ่ง

Constructor = ชื่อเดียวกับ class, **ไม่มี return type** (แม้ void), ทำงานตอน new, ใช้กำหนดค่าเริ่มต้น, overload ได้, **ไม่ถูกสืบทอด** · ไม่เขียนเลย Java ให้ default constructor (ไม่มีพารามิเตอร์) แต่ถ้าเขียนแบบมีพารามิเตอร์แล้ว ตัวเปล่าจะ**หายไป**

this = object ปัจจุบัน (แยก field กับ parameter ชื่อซ้ำ) · `this(...)` เรียก constructor อื่นใน class เดียวกัน (บรรทัดแรก)

Reference = ตัวแปร object เก็บ**ที่อยู่** ไม่ใช่ตัว object · `a = b` → ชี้ object เดียวกัน · `null` = ไม่ชี้อะไร → เรียกเมธอด = NullPointerException · `==` เทียบ reference, `.equals()` เทียบเนื้อหา

static = เป็นของ class ใช้ร่วมกันทุก object เรียกผ่านชื่อ class (`Math.PI`) · static method ใช้ `this` ไม่ได้ · **final**: ตัวแปร=ค่าคงที่ · method=override ไม่ได้ · class=extends ไม่ได้ (เช่น String)

4 เสาหลัก OOP: **Encapsulation** ห่อหุ้ม/ซ่อนข้อมูล (private + getter/setter + validate) · **Inheritance** สืบทอด · **Polymorphism** พ้องรูป · **Abstraction** ซ่อนรายละเอียดแสดงแค่ที่จำเป็น

บท 7 · Array & ArrayList

Array = ตัวแปรชุด เก็บข้อมูล**ชนิดเดียวกัน**หลายค่า เรียงต่อกัน อ้างด้วย **index เริ่ม 0** · **ขนาดคงที่**หลังสร้าง · array เป็น object (ตัวแปรคือ reference)

สร้าง: `int[] a = new int[5];` · `int[] a = {1,2,3};` · `new int[]{1,2,3}` · **ค่าเริ่มต้น** int 0 · double 0.0 · boolean false · char '\u0000' · String/object **null**

length = attribute (ไม่มีวงเล็บ) · index สุดท้าย = length−1 · เกินช่วง/ติดลบ → **ArrayIndexOutOfBoundsException** (runtime ไม่ใช่ compile)

for-each `for(int x : a)` = อ่านทุกตัว ไม่รู้ index แก้ค่าใน array ไม่ได้ · ต้องการ index/แก้ค่า → for ปกติ

2 มิติ = array ของ array `new int[แถว][คอลัมน์]` · `a.length`=จำนวนแถว · `a[0].length`=คอลัมน์ · `a[r][c]` · ลูปนอก=แถว ลูปใน=คอลัมน์ · **Jagged** = แต่ละแถวยาวไม่เท่ากัน → ใช้ `a[r].length`

Array of Objects = เก็บ **reference** · `new Student[3]` สร้างแค่ช่อง (null ×3) ยังไม่มี object ต้อง `new Student(..)` ทีละช่อง · ลืม → **NullPointerException**

ArrayList (`java.util`) = list ขนาด**ยืดหดอัตโนมัติ** · เก็บได้แต่ **object** → primitive ต้องใช้ **Wrapper** (int→Integer, double→Double, char→Character, boolean→Boolean) · Java แปลงให้เอง = **autoboxing/unboxing** · `<Type>` = **Generics** กำหนดชนิด

เมธอด	ทำอะไร
`add(x)` / `add(i,x)`	ต่อท้าย / แทรกที่ i (ตัวหลังเลื่อนขวา)
`get(i)` · `set(i,x)`	อ่าน · แทนที่
`remove(i)` / `remove(obj)`	ลบตาม index / ตามค่า (ตัวหลังเลื่อนซ้าย)
`size()` · `isEmpty()` · `clear()`	จำนวน · ว่างไหม · ลบหมด
`contains(x)` · `indexOf(x)`	มีไหม (boolean) · ตำแหน่ง (ไม่พบ = −1)

× `ArrayList<int>` · `list.length` · `get(size())` · ลบใน for-each (ConcurrentModificationException → วนถอยหลัง) · ลืม import · เทียบ String ด้วย `==` · `i<=a.length`

บท 8 · Inheritance (การสืบทอด)

Inheritance = class ใหม่ (**subclass/child/derived**) รับ attribute+method จาก class เดิม (**superclass/parent/base**) ด้วย `extends` แล้ว**เพิ่ม/แก้**ของตัวเองได้

ประโยชน์: Code reuse · Maintainability · Extensibility · Real-world modeling

IS-A (เป็น...ชนิดหนึ่ง) → ใช้ inheritance · **HAS-A** (มี...) → **composition** (เก็บเป็น attribute) เช่น Car has Engine, Employee has Department · ห้ามใช้ extends กับ has-a

Single inheritance: class extends ได้ **1** class (ไม่มี multiple inheritance ของ class เพื่อเลี่ยง diamond problem) · **Multi-level** ได้: Animal→Dog→Poodle (Poodle ได้ของทั้ง 2 ชั้น) · **Hierarchical**: แม่ 1 ลูกหลายตัว · ทุก class สืบจาก **Object** โดยอัตโนมัติ

access	class	package	subclass	ทุกที่	UML
public	✓	✓	✓	✓	+
protected	✓	✓	✓	×	#
default (ไม่เขียน)	✓	✓	×*	×	~
private	✓	×	×	×	–

*subclass ต่าง package · กว้าง→แคบ: public > protected > default > private

super: `super(args)` เรียก constructor แม่ (**ต้องบรรทัดแรก**) · `super.method()` เรียกเวอร์ชันแม่ · `super.field` · ไม่เขียน → Java แทรก `super()` ให้ → ถ้าแม่ไม่มี constructor เปล่า = **compile error**

ลำดับสร้าง object: new ลูก → constructor แม่ทำงาน**ก่อน**เสมอ (ไล่จากบนสุดลงมา) → แล้วจึงของลูก

Method Overriding = ลูกเขียนเมธอดที่สืบทอดมา**ใหม่** (เปลี่ยนพฤติกรรม) · กฎ: ชื่อเดียวกัน · พารามิเตอร์เหมือนกัน (ชนิด ลำดับ จำนวน) · return type เดียวกันหรือ subtype (covariant) · access **ห้ามแคบลง** · override **ไม่ได้**: static (=method hiding), final, private, constructor

@Override = annotation ไม่บังคับ แต่ให้ compiler ตรวจ ถ้าไม่ได้ override จริง (พิมพ์ชื่อผิด/พารามิเตอร์ผิด) → compile error · ต่อยอดแม่: `super.calculatePay() + bonus` · ไม่เรียก super = แทนที่ทั้งหมด

บท 9 · Abstraction

Abstraction = แสดงเฉพาะสิ่งจำเป็น ซ่อนรายละเอียดการทำงาน (ขับรถรู้แค่พวงมาลัย/คันเร่ง ไม่ต้องรู้ ECU) · ช่วย: กำหนดพฤติกรรมร่วม · ซ่อนส่วนที่ต่าง · ลด coupling · ทำได้ 2 ทาง: **abstract class**, **interface**

Abstract class = class ที่ยังไม่สมบูรณ์ ใช้เป็นแม่ · ใส่ `abstract` · **new ไม่ได้** แต่ใช้เป็นชนิดตัวแปรได้ (`Shape s = new Circle(5)`) · มีได้ทั้ง attribute, **constructor** (ให้ลูกเรียกผ่าน super), เมธอดปกติ, abstract method, static

Abstract method = มีแต่หัว **ไม่มี body** ลงท้าย `;` บังคับลูกให้ override · ห้ามเป็น private / final / static · มี abstract method ≥1 → class **ต้อง** abstract · abstract class มี abstract method **0 ตัว** ก็ได้ (เพื่อกันการ new) · ห้าม `abstract final class`

ลูกต้อง override abstract method **ครบทุกตัว** ไม่งั้นลูกต้องเป็น abstract เอง (Shape → abstract RoundShape → Circle ทำครบ)

Interface = **สัญญา (contract)** กำหนดว่าต้องทำอะไรได้ ไม่บอกวิธี · สร้างด้วย `interface` ใช้ด้วย `implements` · **new ไม่ได้** · **ไม่มี constructor** · ไม่มี instance attribute

ใน interface: method = `public abstract` อัตโนมัติ · ตัวแปร = `public static final` (ค่าคงที่ กำหนดค่าใหม่ไม่ได้) · `default` method มี body (Java 8) · `static` method เรียกผ่านชื่อ interface · `private` method (Java 9)

implements ได้**หลาย** interface (`Phone implements Camera, MusicPlayer, GPS`) · interface `extends` interface ได้หลายตัว · class extends + implements พร้อมกันได้ (**extends ก่อน**) · เมธอดที่ implement **ต้อง public** (ไม่ใส่ = weaker access error) · ต้อง implement ครบ ไม่งั้นต้องเป็น abstract

	Abstract class	Interface
keyword	abstract class / extends	interface / implements
สืบได้	1 class	หลาย interface
constructor	มี	ไม่มี
attribute	ทุกแบบ	ค่าคงที่ public static final
method	abstract + ปกติ	abstract + default/static/private
access	ใดก็ได้	abstract method = public
ความสัมพันธ์	is-a ญาติใกล้ชิด	can-do ความสามารถ (-able)
ใช้เมื่อ	แชร์ field/ctor/โค้ดพื้นฐาน	คลาสไม่เกี่ยวกันมีความสามารถร่วม / ต้องการหลายความสามารถ

บท 10 · Polymorphism (การพ้องรูป)

Polymorphism = Poly (หลาย) + Morph (รูป) · ตัวแปรชนิดเดียวอ้างถึง object ได้หลายชนิด และ**ผลลัพธ์ขึ้นกับ object จริง** · สั่งเหมือนกัน ทำต่างกัน (move: คนเดิน นกบิน ปลาว่าย)

	Compile-time	Runtime
ชื่อ	**Method Overloading** · static/early binding	**Method Overriding** · dynamic method dispatch · late binding
ที่ไหน	class เดียวกัน	แม่–ลูก (ต้องมี inheritance/interface)
พารามิเตอร์	**ต่าง** (จำนวน/ชนิด/ลำดับ)	**เหมือน**
return	ต่างได้ แต่**ต่างแค่ return = error**	เหมือน/covariant
ตัดสินจาก	รายการพารามิเตอร์	ชนิด object จริงตอนรัน (JVM)
จุดประสงค์	รองรับข้อมูลหลายแบบ	เปลี่ยนพฤติกรรมของลูก

กฎทอง: **ชนิดตัวแปร (reference type)** กำหนดว่า**เรียกอะไรได้** (compiler ตรวจ) · **ชนิด object จริง** กำหนดว่า**ทำงานแบบไหน** · `Animal a = new Dog(); a.guardHouse();` → compile error

Upcasting ลูก→แม่ อัตโนมัติ ปลอดภัย · **Downcasting** แม่→ลูก ต้องเขียน `(Dog) a` · ผิดชนิด → compile ผ่าน แต่ **ClassCastException** ตอนรัน · ป้องกันด้วย `instanceof` (คืน boolean, `null instanceof X` = false) · Java 16+: `if (a instanceof Dog d)`

ไม่ polymorphic: **field** (ใช้ตามชนิดตัวแปร: `Parent o=new Child(); o.name`→ของ Parent) · **static** method (hiding) · override ไม่ได้: private, final, constructor

ประโยชน์: โค้ดซ้ำน้อย · ยืดหยุ่น · ขยายง่าย (เพิ่ม Duck ไม่ต้องแก้ลูป) · ลด if-else/switch ตามชนิด · **Loose coupling** · ดูแลง่าย · instanceof/if-else ตามชนิดเยอะ = ใช้ poly ไม่เต็มที่ · **Open/Closed Principle** = เปิดให้ขยาย ปิดการแก้ไข · **Program to an interface**: รับพารามิเตอร์เป็นชนิดแม่/interface `checkout(PaymentMethod p)`

วิธีไล่ผลการรัน

① new ลูก → พิมพ์ของ constructor แม่ก่อน ② เรียกเมธอด → ชนิดตัวแปรมีไหม (ไม่มี = compile error) ③ มี → ใช้เวอร์ชันของ object จริง (ถ้าลูก override) ④ เจอ super.m() → รันของแม่แล้วกลับมาต่อ ⑤ field ใช้ตามชนิดตัวแปร

`"a"+1+2`="a12" · `1+2+"a"`="3a" · `print` ไม่ขึ้นบรรทัด · `printf("%.2f%n")` ทศนิยม 2 ตำแหน่ง · `10/0` Exception แต่ `10.0/0`=Infinity · Circle r=5 → 78.54

โจทย์ถาม → ตอบ (แนวคิด/ออกแบบ)

ถ้าโจทย์ถาม...	ตอบ
กันไม่ให้ override / ไม่ให้สืบทอด	**final method / final class**
class ที่ new ไม่ได้ ใช้เป็นแม่แบบ	**abstract class**
เมธอดไม่มี body บังคับลูกเขียน	**abstract method**
ให้ลูกเข้าถึงได้แต่คนนอกไม่ได้	**protected (#)**
ค่าคงที่ใช้ร่วมกันทุก object	**static final**
เพิ่มคลาสใหม่ได้โดยไม่แก้โค้ดเดิม	**Open/Closed Principle**
wrapper ของ int / char	**Integer / Character**
จำนวนแถว / คอลัมน์ array 2 มิติ	**a.length / a[0].length**
Circle, Rectangle มี color ร่วม แต่สูตรพื้นที่ต่างกัน	**abstract class Shape**
Bird กับ Airplane ไม่เกี่ยวกันแต่บินได้ทั้งคู่	**interface Flyable**
จ่ายเงินหลายช่องทาง เพิ่มช่องทางใหม่ง่าย	**interface PaymentMethod + polymorphism**

แม่แบบข้อเขียน 2.1 (UML → โค้ด)

```
public abstract class Shape {  // «abstract»
  protected String color;      // # = protected
  public Shape(String color) {
    this.color = color;
  }
  public abstract double calculateArea();//ไม่มี{}
}
public class Triangle extends Shape {// ▷
  private double base;         // - = private
  private double height;
  public Triangle(String color, double base,
    double height) {
    super(color);              // ของแม่
    this.base = base;          // ของเรา
    this.height = height;
  }
  @Override
  public double calculateArea() {
    return 0.5 * base * height;
  }
}
```

constructor = ชื่อเหมือนคลาส ไม่มีชนิดหน้าชื่อ · UML `name : String` → `String name;` · `getName() : String` → `public String getName()`

สูตร วงกลม `Math.PI * r * r` · สี่เหลี่ยม `w * h` · สามเหลี่ยม `0.5 * b * h` (ห้าม `1/2` = 0) · FullTime `salary + bonus` · PartTime `hours * rate` · จำไม่ได้ → เขียนคอมเมนต์ + return ไว้

หน้า 2 · บท 11–12 · import · หลักการ · ต่างกันอย่างไร

เช็คจุดที่พลาดบ่อย (ดูก่อนส่ง)

1 บล็อกแรก = **try** · `catch (ชื่อError e)` มีวงเล็บ · **finally** ไม่มีวงเล็บ ทำเสมอ

2 ในวงเล็บ catch ใส่**ชื่อ Exception** ไม่ใช่ method: parse → **NumberFormatException** · หาร → **ArithmeticException** · nextInt → **InputMismatchException**

3 `btn.____(e -> …)` = **addActionListener** · อ่าน = **getText** · แสดง = **setText**

4 abstract method → ลงท้าย **;** ไม่มี `{}` · **new คลาส abstract ไม่ได้** (compile error) แต่ `Shape s = new Circle()` ได้

5 constructor ลูก บรรทัดแรก **super(…)** ไม่ใช่ return · **@Override** อยู่หน้า method ชื่อเหมือนแม่เป๊ะ · อย่าลืมคำว่า **class**

6 ตัวเล็กหมด: `try catch finally private protected public class extends new` · ตัวใหญ่: `JTextField JButton JLabel System String` · method มี **()** เสมอ · `{ }` ครบคู่

7 JCheckBox ☑ หลายข้อ · JRadioButton ◉ ข้อเดียว + ButtonGroup

บท 11 · Exception Handling

Exception = เหตุการณ์ผิดปกติ**ขณะรัน** ที่ขัดจังหวะการทำงานปกติ · ไม่จัดการ → โปรแกรมหยุด บรรทัดหลังไม่ทำ แสดง `Exception in thread "main" java.lang.ArithmeticException: / by zero`

ข้อผิดพลาด 3 แบบ: **Syntax/Compile error** ผิดไวยากรณ์ (ลืม ; ชื่อผิด) compile ไม่ผ่าน · **Runtime error** รันแล้วพัง (หาร 0, index เกิน) ← Exception · **Logical error** รันได้ผลผิด (สูตรผิด) ไม่มี error แจ้ง

Exception handling = ไม่ได้ทำให้ error หายไป แต่ทำให้โปรแกรม**รู้วิธีรับมือ** และทำงานต่อได้

Exception	สาเหตุ
ArithmeticException	หาร **int** ด้วย 0 ("/ by zero")
ArrayIndexOutOfBounds Exception	index <0 หรือ ≥ length
NumberFormatExceptio n	แปลง String ที่ไม่ใช่ตัวเลข parseInt("ABC"), parseDouble("")
NullPointerException	ใช้ reference ที่เป็น null
InputMismatchExceptio n	Scanner nextInt() แต่ผู้ใช้พิมพ์อักษร (java.util)
ClassCastException	downcast ผิดชนิด
IllegalArgumentExceptio n	อาร์กิวเมนต์ไม่ถูกต้อง (มัก throw เอง)
FileNotFound / IOException	เปิด/อ่านไฟล์ไม่ได้ (**checked**)

ลำดับชั้น: **Throwable** → **Error** (ร้ายแรงระดับระบบ เช่น OutOfMemoryError, StackOverflowError ไม่ควรจับ) | **Exception** → **RuntimeException** (unchecked) + อื่น ๆ (checked) · `catch(Exception e)` จับลูกทุกตัวได้เพราะ polymorphism

Unchecked = RuntimeException และลูก (รวม Error) · compiler **ไม่บังคับ**จัดการ · มักเกิดจากบั๊กโปรแกรมเมอร์ · **Checked** = Exception ที่ไม่ใช่ Runtime · compiler **บังคับ** try-catch หรือ throws ไม่งั้น compile error

try = โค้ดที่อาจเกิด exception · **catch** = จัดการเมื่อเกิด (เลือกตัว**แรก**ที่ชนิดตรง ทำตัวเดียว) · **finally** = **ทำเสมอ** ไม่ว่าเกิดหรือไม่ (ปิดไฟล์/คืนทรัพยากร) · try ต้องตามด้วย catch หรือ finally อย่างน้อย 1

Flow: try ปกติ → ข้าม catch → finally → ต่อ · เกิด exception → **ข้ามบรรทัดที่เหลือใน try** → catch ที่ตรง → finally → ต่อ · ไม่มี catch ตรง → finally → โปรแกรมพัง

Multiple catch: เรียง**เจาะจง → กว้าง** (ลูกก่อนแม่) · Exception ไว้ท้ายสุด · สลับ = compile error (unreachable) · `catch(A | B e)` จับหลายชนิดในบล็อกเดียว

ตัวแปร e: `e.getMessage()` → "/ by zero" · `System.out.println(e)` → "java.lang.ArithmeticException: / by zero" · `e.printStackTrace()` → debug

throw = **โยน** exception ทันที ในเนื้อเมธอด `throw new IllegalArgumentException("Age cannot be negative");` (ใช้ป้องกันข้อมูลผิด = encapsulation) · **throws** = **ประกาศ**ที่หัวเมธอดว่าอาจโยน ให้ผู้เรียกจัดการ `void readFile() throws IOException`

Custom exception: `class InsufficientBalanceException extends Exception` (checked) หรือ extends RuntimeException (unchecked) · constructor `super(message)` · ใช้: `if(amount>balance) throw new …("Insufficient balance");` + ประกาศ `throws`

```
try { int[] x={10,20,30}; println(x[5]); int
  r=10/0; }
catch(ArithmeticException e){println("A");}
catch(ArrayIndexOutOfBoundsException e)
  {println("I");}
finally{println("F");}  println("End");    // I F
  End
Scanner: 10,2→Result = 5 · 10,0→Cannot divide by
  zero
        ABC→Please enter a number (ไม่ถามเลขที่ 2)
```

บท 12 · Java GUI (Swing)

GUI = ติดต่อผู้ใช้ผ่านหน้าต่าง ปุ่ม ช่องกรอก (ต่างจาก **CLI**/console ที่ใช้ Scanner) · **AWT** = ชุดแรก (java.awt, heavyweight ใช้ของ OS) · **Swing** = พัฒนาจาก AWT (javax.swing, ขึ้นต้น J, component มากกว่า, มีใน JDK, OOP + event-driven) · **JavaFX** = ใหม่ รองรับ CSS/animation

method GUI ที่ต้องใช้ (เรียงตามลำดับเขียนโค้ด)

method	ทำอะไร
`class X extends JFrame`	หน้าต่างของเรา "เป็น" JFrame
`setTitle("…")` · `setSize(w,h)`	ตั้งชื่อหน้าต่าง · ขนาด กว้าง,สูง
`setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)`	กดปิดแล้วโปรแกรมจบ
`new JButton("…")` `new JTextField(15)` `new JLabel("…")`	สร้างปุ่ม · ช่องพิมพ์ · ป้ายข้อความ
`setLayout(new FlowLayout())`	เลือกวิธีจัดเรียงของในหน้าต่าง/panel
`add(x)` · `add(x, BorderLayout.NORTH)`	วางของลงหน้าต่าง (ไม่ add = มองไม่เห็น)
`btn.addActionListener(e -> …)`	ผูกปุ่มกับงาน ทำเมื่อถูกคลิก
`field.getText()`	อ่านสิ่งที่ผู้ใช้พิมพ์ (ได้ String)
`Integer.parseInt(…)` / `Double.parseDouble(…)`	แปลงข้อความเป็นตัวเลข (ผิด → NumberFormatException)
`label.setText("…")`	เปลี่ยนข้อความที่แสดงบนจอ
`JOptionPane.showMessageDialog(this,"…")`	เด้งกล่องแจ้งเตือน
`setLocationRelativeTo(null)`	หน้าต่างอยู่กลางจอ
`new X().setVisible(true)`	แสดงหน้าต่าง (ทำท้ายสุดใน main)

component ไหนทำอะไร

component	ทำอะไร / method ที่ใช้
JFrame	หน้าต่างหลัก (คลาสเรา extends JFrame)
JPanel	กล่องจัดกลุ่มของ ใส่ layout ของตัวเองได้
JLabel	ข้อความ/รูปที่แสดง ผู้ใช้แก้ไม่ได้ · `setText`
JTextField	ช่องพิมพ์ **1 บรรทัด** · `getText` / `setText`
JPasswordField	ช่องรหัสผ่าน (ขึ้น ●●●)
JTextArea	ช่องพิมพ์**หลายบรรทัด** (ใส่ใน JScrollPane)
JButton	ปุ่ม · `addActionListener`
JCheckBox ☑	เลือกได้**หลายข้อ** · `isSelected()`
JRadioButton ◉	เลือกได้**ข้อเดียว** ต้องใส่ **ButtonGroup** · `isSelected()`
JComboBox	dropdown รายการให้เลือก · `getSelectedItem()`
JTable	ตาราง (ใส่ใน JScrollPane)
JScrollPane	แถบเลื่อน ห่อ TextArea/Table
JOptionPane	กล่อง popup แจ้งเตือน/ถาม/รับค่า
JMenuBar → JMenu → JMenuItem	แถบเมนูด้านบน

Layout Manager = กฎจัดตำแหน่ง/ขนาด component · **FlowLayout** เรียงซ้าย→ขวา เต็มขึ้นบรรทัดใหม่ (ค่าเริ่มต้นของ **JPanel**, ฟอร์มเล็ก) · **BorderLayout** 5 ส่วน NORTH/SOUTH/EAST/WEST/CENTER (ค่าเริ่มต้นของ **JFrame**, ไม่ระบุ = CENTER, ใส่ซ้ำช่องเดิม = ทับ) · **GridLayout(r,c,hgap,vgap)** ตารางช่องเท่ากัน เติมทีละแถว · **BoxLayout** แนวเดียว · **GridBagLayout** ยืดหยุ่นสุด · ใช้ JPanel ผสมหลาย layout · **ไม่แนะนำ null layout + setBounds** (ไม่ยืดหยุ่น เปลี่ยนเครื่อง/ฟอนต์แล้วเพี้ยน)

Event-Driven Programming = โปรแกรม**รอ**ผู้ใช้กระทำ (Event) แล้วตอบสนอง ไม่ได้รันบนลงล่างจบ · Action → Listen → Respond · **Event Source** (JButton) → **Event Object** (ActionEvent) → **Event Listener** (ActionListener)

Listener	ใช้กับ / เมธอด
ActionListener	คลิกปุ่ม/Enter/เมนู · `actionPerformed(ActionEvent e)` · มีเมธอดเดียว → ใช้ **lambda** `e -> …`

Adapter class = override เฉพาะเมธอดที่ใช้ · ไม่ `addActionListener` = กดแล้วเงียบ

JOptionPane: `showMessageDialog` แจ้งเตือน (void) · `showInputDialog` รับข้อความ คืน **String / null ถ้ากด Cancel** · `showConfirmDialog(…, YES_NO_OPTION)` คืน **int** เทียบ `YES_OPTION` · ชนิดไอคอน: INFORMATION · WARNING · ERROR · QUESTION · PLAIN _MESSAGE

ค่าที่ได้อื่น ๆ: `isSelected()` (check/radio) · `(String) getSelectedItem()` (combo) · `setEditable(false)` · `requestFocus()` · **DefaultTableModel** addRow/removeRow, `getSelectedRow()` = −1 ถ้าไม่เลือก

ต้อง import อะไร (ออกแค่ 2–3 ข้อ)

import	ใช้กับ
`java.util.` …	**ArrayList** · **Scanner** · InputMismatchException
`javax.swing.` …	ทุกตัวขึ้นต้น **J** (JFrame JButton JLabel JTextField …) · JOptionPane · ButtonGroup
`java.awt.` …	Layout: **FlowLayout** · **BorderLayout** · **GridLayout** · Color · Font
`java.awt.event.` …	**ActionListener** · **ActionEvent**
`javax.swing.table.` …	DefaultTableModel
`java.io.` …	IOException · FileReader
ไม่ต้อง import	String · Math · Integer · Double · System · Exception · Arithmetic/NullPointer/NumberFormat/ArrayIndexOutOfBounds Exception

ตัวอย่างแบบเต็ม: `import java.util.ArrayList;` `import java.awt.event.ActionListener;` `import javax.swing.JFrame;`

ช้อยส์หลอก javax.swing.ActionListener × · java.awt.JOptionPane × · javax.swing.BorderLayout × · java.io.Scanner × · java.lang.ArrayList ×

โค้ดแบบนี้ = หลักการอะไร

เห็นโค้ด	คือ
`private` field + getter/setter (มี if ตรวจ)	**Encapsulation**
ลูกมีเมธอดชื่อ+พารามิเตอร์เหมือนแม่ / `@Override`	**Overriding (runtime poly)**
class เดียวมี `add(int,int)` `add(double,double)`	**Overloading (compile-time poly)**
constructor หลายตัวพารามิเตอร์ต่างกัน	**Constructor overloading**
`Animal a = new Dog();` / `(Dog) a`	**Upcasting / Downcasting**
`for(Animal a : zoo) a.makeSound();`	**Polymorphism + dynamic binding**
class มี field เป็น object อื่น `private Engine engine;`	**Composition (HAS-A)**
`class MyEx extends Exception`	**Custom (checked) exception**
`btn.addActionListener(e -> …)`	**Event handling + Lambda**
`new ActionListener(){…}` / `new MouseAdapter(){…}`	**Anonymous class / Adapter**
`ArrayList<Integer> list.add(5)`	**Generics + Autoboxing**

คู่ที่ชอบถามว่า "ต่างกันอย่างไร"

คู่	ต่างกัน
Overload / Override	class เดียว พารามิเตอร์ต่าง ตัดสินตอน compile / แม่-ลูก พารามิเตอร์เหมือน ตัดสินตอน runtime
extends / implements	สืบ class ได้ 1 / ทำตาม interface ได้หลายตัว
this / super	object ตัวเอง / ส่วนของคลาสแม่
IS-A / HAS-A	inheritance / composition (เก็บเป็น field)
Upcast / Downcast	ลูก→แม่ อัตโนมัติ / แม่→ลูก ต้อง cast เสี่ยง ClassCastException
Abstraction / Encapsulation	ซ่อนวิธีทำ บอกแค่ทำอะไร / ซ่อนข้อมูล ควบคุมการเข้าถึง
abstract class / interface	มี ctor+field สืบได้ 1 / สัญญา ค่าคงที่ implements หลายตัว
== / equals()	เทียบ reference (ที่อยู่) / เทียบเนื้อหา
static / instance	ของ class ใช้ร่วมกัน / ของแต่ละ object
throw / throws	โยนทันทีในเนื้อเมธอด / ประกาศที่หัวเมธอด
final / finally	ห้ามแก้/override/สืบทอด / บล็อกที่ทำเสมอ (ก่อน return ด้วย)
Checked / Unchecked	compiler บังคับ (IOException) / ไม่บังคับ (RuntimeException)
JCheckBox / JRadioButton	เลือกได้หลายข้อ ☑ / เลือกข้อเดียว ◉ (+ButtonGroup)
JTextField / JTextArea	พิมพ์ 1 บรรทัด / หลายบรรทัด
Array / ArrayList	ขนาดคงที่ .length / ยืดหด .size()
Compile / Runtime error	ผิดไวยากรณ์ รันไม่ได้ / รันแล้วพัง
Listener / Adapter	ต้อง override ทุกเมธอด / override เฉพาะที่ใช้